NLP CS60075 Assignment II — Datasets

How to read this document. Every dataset used anywhere across the 11 projects has one entry here: where to get it, its exact file/field format, its official splits (or how to construct one if it has none), any preprocessing you must do before using it, and – separately for every task that uses it – exactly how it is meant to be used in that task. A dataset reused by more than one project (e.g. UD English EWT, used in Projects 2, 3, and 5) is documented in full only once, at its first appearance; every later reuse is a short entry that points back to that first entry and states only what is genuinely different about how the later project uses the same file. Section numbers below match the "companion Datasets document, Section N" references already given in the Projects document.

1 Project 1 Datasets: N-gram LM, Smoothing, and Neural LM Reproduction

Datasets 1.1–1.3 below are the three *assignable* datasets – your group works with whichever one you are assigned, using it for all three tasks. Dataset 1.B (BLiMP) is a separate, shared bonus check that every group runs identically, regardless of which of 1.1–1.3 they were assigned.

Dataset 1.1 — WikiText-2

Source

`https://huggingface.co/datasets/Salesforce/wikitext`, configuration `wikitext-2-raw-v1`. Load with `load_dataset("Salesforce/wikitext", "wikitext-2-raw-v1")`. License: CC-BY-SA-3.0.

Format

A single text column. Each example is one line of raw Wikipedia article text as it appears in the article, including section-heading lines (wrapped in `= =`) and blank lines separating articles.

Field	Type	Meaning
`text`	string	One line of raw article text, a heading line, or an empty string (article separator).

Splits

Split	Size	Notes
`train`	≈2.09M tokens	Use for all training in Tasks 1–3.
`validation`	≈218K tokens	Hyperparameter tuning only (k, λ, checkpoint selection). Never touch this for final numbers.
`test`	≈246K tokens	Final reported perplexity only.

Required Preprocessing

1. Concatenate all `text` lines within a split into one long token stream – WikiText's lines are a storage artifact, not sentence boundaries.
2. Tokenize with a punctuation-aware whitespace tokenizer and lowercase every token.
3. Build the vocabulary from `train` only: keep the 30,000 most frequent tokens; map every other token to `<UNK>`. Tokens occurring < 2 times in train also map to `<UNK>`.

Task 1 use: If assigned this dataset: train n-gram count tables on `train`; tune smoothing hyperparameters on `validation`; report final perplexity on `test`.

Task 2 use: If assigned this dataset: train the AWD-LSTM on `train`; use `validation` for checkpoint selection if deviating from a fixed epoch count; report final perplexity on `test`.

Task 3 use: If assigned this dataset: no additional training – reuse the Task 1/2 models. Use `validation` only to tune λ; report final perplexity on `test`.

Dataset 1.2 — DailyDialog

Source

`https://huggingface.co/datasets/daily_dialog`. Load with `load_dataset("daily_dialog")`.

Format

Each example is one multi-turn dialogue.

Field	Type	Meaning
`dialog`	list[string]	The utterances of the dialogue, in speaker-alternating order.
`act`	list[int]	Dialogue-act label per utterance. **Not used in this project.**
`emotion`	list[int]	Emotion label per utterance. **Not used in this project.**

Splits

Split	Size	Notes
`train`	11,118 dialogues	Use for all training in Tasks 1–3.
`validation`	1,000 dialogues	Hyperparameter tuning only.
`test`	1,000 dialogues	Final reported perplexity only.

Required Preprocessing

1. For every dialogue in each split, join the `dialog` list into one document: concatenate utterances in order, separated by a single space – do not insert speaker tags; your model is never trained with any such tokens.
2. Tokenize/lowercase with the same convention as Dataset 1.1, but build the vocabulary *from this dataset's own `train` split* – unlike a cross-dataset evaluation setup, each assignable dataset here is self-contained, so DailyDialog's vocabulary should come from DailyDialog, not from WikiText-2.

Task 1 use: If assigned this dataset: train n-gram count tables on `train`; tune smoothing hyperparameters on `validation`; report final perplexity on `test`.

Task 2 use: If assigned this dataset: train the AWD-LSTM on `train`; use `validation` for checkpoint selection; report final perplexity on `test`.

Task 3 use: If assigned this dataset: no additional training – reuse the Task 1/2 models. Use `validation` only to tune λ; report final perplexity on `test`.

Dataset 1.3 — Gutenberg (NLTK)

Source

Bundled with NLTK: `nltk.download('gutenberg')`, then `nltk.corpus.gutenberg`. A small collection of 18 complete public-domain English texts (e.g. Austen's *Emma*, Melville's *Moby Dick*, several Shakespeare plays), ≈2.6M words total.

Format

`nltk.corpus.gutenberg.raw(fileid)` returns the full raw text of one work; `nltk.corpus.gutenberg.fileids()` lists all 18 file identifiers (e.g. `austen-emma.txt`).

Field	Type	Meaning
(no tabular fields — each item is one complete literary text, not a structured record)		

Splits

Split	Size	Notes
all 18 texts	≈2.6M words	No official split; construct your own by *whole file*, not by token position (next column).

Required Preprocessing

1. Split by file, not by concatenating everything and cutting at a token count: shuffle the 18 file IDs once with a fixed seed, and assign roughly 80% of *files* to train, 10% to validation, 10% to test. Splitting by whole file (rather than slicing one continuous stream) avoids near-duplicate context leaking across splits, since different chapters of the same novel are highly self-similar in vocabulary and style.
2. Concatenate the raw text of each split's assigned files into one stream; tokenize/lowercase with the same convention as Dataset 1.1; build the vocabulary from this dataset's own train split.
3. Gutenberg's 19th-century prose includes archaic spelling and punctuation conventions (e.g. long dashes, unusual capitalization) – do not silently strip these as "errors"; treat them as ordinary tokens, the same as any other word.

Task 1 use: If assigned this dataset: train n-gram count tables on your constructed train split; tune smoothing hyperparameters on validation; report final perplexity on test.

Task 2 use: If assigned this dataset: train the AWD-LSTM on your constructed train split; use validation for checkpoint selection; report final perplexity on test.

Task 3 use: If assigned this dataset: no additional training – reuse the Task 1/2 models. Use validation only to tune λ; report final perplexity on test.

Dataset 1.B (Bonus, shared across every group) — BLiMP

Source

`https://huggingface.co/datasets/nyu-mll/blimp` (Warstadt et al., 2020). Load with `load_dataset("nyu-mll/blimp", "<paradigm_name>")` – BLiMP is 67 separate paradigm configs, so each selected paradigm is loaded as its own dataset object.

Format

Each example is one *minimal pair*: a grammatical sentence and an ungrammatical sentence differing by one targeted phenomenon.

Field	Type	Meaning
`sentence_good`	string	The grammatical sentence.
`sentence_bad`	string	The minimally-different ungrammatical sentence.
`field`	string	Broad category (`morphology`, `syntax`, `semantics`).
`UID`	string	The specific paradigm name (e.g. `wh_island`).

Splits

One split per paradigm, 1,000 pairs each, no train/validation/test division – a pure evaluation benchmark. This dataset is **not tied to your group's assigned dataset above** – every group evaluates on BLiMP identically, regardless of whether they were assigned WikiText-2, DailyDialog, or Gutenberg for Datasets 1.1–1.3.

Required Preprocessing

1. Select 8–10 paradigms spanning all three `field` categories – not clustered in one. A reasonable starting set: `anaphor_gender_agreement`, `determiner_noun_agreement_1` (morphology); `wh_island`, `npi_present_1` (syntax); `principle_A_domain_1`, `drop_argument` (semantics) – add 2–4 more of your choosing to reach 8–10, and use the identical selection across Tasks 1, 2, and 3.
2. Tokenize both sentences with the identical tokenizer/vocabulary as whichever dataset your group was assigned (1.1, 1.2, or 1.3), mapping OOV to `<UNK>`.
3. **Length normalization:** always divide each sentence's total log-probability by its own token count before comparing the pair.

Task 1 use: For your best smoothing variant, compute length-normalized log-probability of both sentences per pair; accuracy = fraction where the model prefers `sentence_good`.

Task 2 use: Identical protocol to Task 1, applied to the trained AWD-LSTM.

Task 3 use: Identical protocol to Tasks 1–2, applied to the Task 3 hybrid's interpolated probabilities.

2 Project 2 Datasets: Statistical vs. Neural Sequence Labeling

Dataset 2.1 — UD English EWT

Source

`https://huggingface.co/datasets/universal_dependencies`, configuration `en_ewt`. Load with `load_dataset("universal_dependencies", "en_ewt")`. This treebank's schema is reused, unchanged, by Projects 4 and 5 – documented fully here once.

Format

Each example is one sentence in CoNLL-U format: per-token POS and dependency-parse annotation (only the POS fields are needed for this project; the dependency fields become relevant when this same schema is reused in Projects 4–5).

Field	Type	Meaning
`tokens`	list[string]	The raw word forms of the sentence, in order.
`upos`	list[int]	Universal POS tag per token (17-tag set) – the label used in this project.
`head`	list[string]	Index of each token's syntactic head. Used in Projects 4–5, not here.
`deprel`	list[string]	Dependency relation to that head. Used in Projects 4–5, not here.

Splits

Split	Size	Notes
`train`	≈12,500 sentences	Train the HMM/BiLSTM-CRF here.
`validation`	≈2,000 sentences	Early stopping / hyperparameter selection (Task 2).
`test`	≈2,000 sentences	Final reported accuracy only.

Required Preprocessing

1. Use the `tokens`/`upos` fields as given – do not re-tokenize.
2. Map integer `upos` codes to their 17 string tag names using the dataset's own `ClassLabel` feature (`dataset.features['upos'].feature.names`).

Task 1 use: If assigned this dataset: train the HMM's transition/emission tables on `train`; evaluate accuracy on `test`.

Task 2 use: If assigned this dataset: train the BiLSTM-CRF on `train`, using `validation` for early stopping; evaluate on `test`.

Task 3 use: If assigned this dataset: retrain the hybrid on `train`; evaluate on `test`.

Dataset 2.2 — UD English GUM

Source

`https://huggingface.co/datasets/universal_dependencies`, configuration `en_gum`. Load with `load_dataset("universal_dependencies", "en_gum")`. Schema reused, unchanged, by Project 4.

Format

Identical CoNLL-U schema to Dataset 2.1 – same annotation scheme, different source genres (academic writing, fiction, spoken transcripts, how-to guides, rather than EWT's web text).

Splits

Split	Size	Notes
`train`	≈2,300 sentences	Train here (this dataset is fully self-contained, like Dataset 2.1 – it is not an evaluation-only add-on).
`validation`	≈780 sentences	Early stopping / hyperparameter selection (Task 2).
`test`	≈780 sentences	Final reported accuracy only.

Required Preprocessing

1. Apply the identical `upos` label mapping used for Dataset 2.1 (Universal POS tags are shared across all UD treebanks).

Task 1 use: If assigned this dataset: train the HMM on `train`; evaluate accuracy on `test`.

Task 2 use: If assigned this dataset: train the BiLSTM-CRF on `train`, using `validation` for early stopping; evaluate on `test`.

Task 3 use: If assigned this dataset: retrain the hybrid on `train`; evaluate on `test`.

Dataset 2.3 — CoNLL-2003

Source

`https://huggingface.co/datasets/eriktks/conll2003`. Load with `load_dataset("eriktks/conll2003")`. Schema reused, unchanged, by Project 9.

Format

Each example is one sentence with per-token named-entity tags in IOB2 format.

Field	Type	Meaning
`tokens`	list[string]	The raw word forms of the sentence, in order.
`ner_tags`	list[int]	IOB2 entity tag per token, 9 classes: `O`, `B-/I-PER`, `B-/I-ORG`, `B-/I-LOC`, `B-/I-MISC`.
`pos_tags`	list[int]	Penn-Treebank-style POS tag per token. **Not used in this project.**

Splits

Split	Size	Notes
`train`	14,041 sentences	Train here.
`validation`	3,250 sentences	Hyperparameter/checkpoint selection (Task 2).
`test`	3,453 sentences	Final reported entity-level F1 only.

Required Preprocessing

1. Map integer `ner_tags` to their 9 string labels via the dataset's own `ClassLabel` feature.
2. Entity-level F1 (not token-level accuracy) requires scoring whole predicted spans against whole gold spans – use `seqeval` rather than hand-rolling span-matching logic.

Task 1 use: If assigned this dataset: your project becomes a named-entity-recognition task rather than POS

tagging. Train the HMM on `train` over the 9 NER tags; evaluate entity-level F1 on `test`.
Task 2 use: If assigned this dataset: train the BiLSTM-CRF on `train`; evaluate entity-level F1 on `test`.
Task 3 use: If assigned this dataset: retrain the hybrid on `train`; evaluate entity-level F1 on `test`.

3 Project 3 Datasets: From CKY to Neural Grammar Induction

All three datasets below come from the same underlying source (the NLTK Penn Treebank sample) split into three disjoint partitions, so that each group gets its own independent set of gold trees. The construction procedure is described once, under Partition A; Partitions B and C follow the identical procedure with a different third of the data.

Dataset 3.1 — Penn Treebank, Partition A

Source

Bundled with NLTK: `nltk.download('treebank')`, then `nltk.corpus.treebank`. This is the freely-redistributable 10% sample of the full (licensed) Penn Treebank, 3,914 sentences total, with full gold constituency parses.

Format

Each example is one sentence's full constituency parse, in Penn Treebank bracket notation, e.g. `(S (NP (DT the) (NN dog)) (VP (VBZ runs)))`. `nltk.corpus.treebank.parsed_sents()` returns these already parsed into NLTK `Tree` objects.

Splits

Split	Size	Notes
Partition A / `train`	≈1,043 sentences	Roughly 80% of this partition's third.
Partition A / `validation`	≈130 sentences	Roughly 10%.
Partition A / `test`	≈131 sentences	Roughly 10%.

Required Preprocessing

1. **Construct all three partitions with one shared procedure** (do this once; every group in this project should use the identical procedure with the identical seed, so partitions are reproducible and disjoint): shuffle all 3,914 sentence indices once with a fixed seed (e.g. `numpy.random.seed(42)`), then cut the shuffled list into three equal thirds – indices [0:1305) = Partition A, [1305:2610) = Partition B, [2610:3914) = Partition C.
2. Within Partition A, further split 80/10/10 into train/validation/test (a second, independent shuffle within the partition, same fixed seed) – save all resulting sentence-index lists to a file so the exact same split is reused across Tasks 1 and 3.
3. Strip functional tags and trace indices from non-terminal labels (e.g. treat `NP-SBJ-1` as plain `NP`).

Task 1 use: If assigned this partition: train the CKY+PCFG pipeline on Partition A's train trees; evaluate bracketing F1 on Partition A's test trees.
Task 2 use: If assigned this partition: train the compound PCFG unsupervised on Partition A's train-split *sentences only* (trees stripped); evaluate the induced grammar's bracketing F1 against Partition A's gold test trees.
Task 3 use: If assigned this partition: train the supervised neural PCFG on Partition A's train trees; evaluate bracketing F1 on Partition A's test trees.

Dataset 3.2 — Penn Treebank, Partition B

Source, Format, Splits, Preprocessing

Identical to Dataset 3.1's construction procedure (Section 3) – not repeated here. This is the second third of the same shuffled split (indices [1305:2610)), further divided 80/10/10 into its own train/validation/test.
Task 1 use: If assigned this partition: train the CKY+PCFG pipeline on Partition B's train trees; evaluate bracketing F1 on Partition B's test trees.
Task 2 use: If assigned this partition: train the compound PCFG unsupervised on Partition B's train-split sentences only; evaluate against Partition B's gold test trees.
Task 3 use: If assigned this partition: train the supervised neural PCFG on Partition B's train trees; evaluate on Partition B's test trees.

Dataset 3.3 — Penn Treebank, Partition C

Source, Format, Splits, Preprocessing

Identical to Dataset 3.1's construction procedure (Section 3) – not repeated here. This is the remaining third of the same shuffled split (indices [2610:3914)), further divided 80/10/10 into its own train/validation/test.

Task 1 use: If assigned this partition: train the CKY+PCFG pipeline on Partition C's train trees; evaluate bracketing F1 on Partition C's test trees.

Task 2 use: If assigned this partition: train the compound PCFG unsupervised on Partition C's train-split sentences only; evaluate against Partition C's gold test trees.

Task 3 use: If assigned this partition: train the supervised neural PCFG on Partition C's train trees; evaluate on Partition C's test trees.

4 Project 4 Datasets: Extractive vs. Abstractive Text Summarization

Dataset 4.1 — CNN/DailyMail (3.0.0)

Source

`https://huggingface.co/datasets/abisee/cnn_dailymail`, configuration `3.0.0`. Load with `load_dataset("abisee/cnn_dailymail", "3.0.0")`.

Format

Each example is one news article paired with a human-written, multi-sentence summary (the original dataset calls this field `highlights`) that has relatively high overlap with the source – many summary sentences are close paraphrases of individual article sentences, which is exactly why extractive selection is a reasonable strategy on this dataset.

Field	Type	Meaning
`article`	string	The full news article text.
`highlights`	string	The gold summary: several sentences, separated by newlines within the string.
`id`	string	Article identifier. Not a modeling feature.

Splits

Split	Size	Notes
`train`	≈287,000 articles	Fit TF-IDF/IDF statistics and any tuned thresholds here.
`validation`	≈13,000 articles	Tune k (Task 1), hyperparameters (Task 2), and m (Task 3) here.
`test`	≈11,000 articles	Final reported metrics only.

Required Preprocessing

1. Sentence-split `article` with a standard sentence tokenizer (e.g. `nltk.sent_tokenize` or spaCy's sentencizer) – this is the unit both TextRank/LexRank and ROUGE operate over.
2. Split `highlights` on its internal newlines to recover the gold summary as a list of sentences (needed for computing extractive density and compression ratio against a multi-sentence reference).
3. Some articles retain source boilerplate (e.g. a leading `(CNN) --` byline marker) – strip this before sentence-splitting, or it can end up selected as a spurious high-centrality "sentence" by TextRank/LexRank.

Task 1 use: If assigned this dataset: build the TF-IDF sentence graph from `train`; tune k (and LexRank's edge-weight cutoff) on `validation`; evaluate on `test`.

Task 2 use: If assigned this dataset: train the pointer-generator on `train` (50k-word vocabulary built from `train` only); evaluate on `test` with Task 1's exact metrics.

Task 3 use: If assigned this dataset: tune m on `validation`; run Task 1's extractor into Task 2's trained model on `test`; evaluate ROUGE and the factual-consistency proxy against Task 1 and Task 2.

Dataset 4.2 — XSum

Source

`https://huggingface.co/datasets/EdinburghNLP/xsum`. Load with `load_dataset("EdinburghNLP/xsum")`.

Format

Each example is one BBC article paired with a single-sentence, professionally-written summary that is *genuinely* abstractive: it is written to be a one-line introduction to the story, not a compressed restatement of any one article sentence, so it typically shares very few exact words with the source. This makes XSum a meaningfully harder setting for Task 1's extractive approach than Dataset 4.1 – expect lower ROUGE for TextRank/LexRank here, and treat that as

an expected property of the dataset, not a bug in your implementation.

Field	Type	Meaning
`document`	string	The full article text.
`summary`	string	The gold summary: exactly one sentence.
`id`	string	Article identifier. Not a modeling feature.

Splits

Split	Size	Notes
`train`	≈204,000 articles	Fit TF-IDF/IDF statistics here.
`validation`	≈11,000 articles	Tune k, hyperparameters, and m here – because the gold summary is only one sentence, expect the best k found on this split to be small (possibly k=1).
`test`	≈11,000 articles	Final reported metrics only.

Required Preprocessing

1. Sentence-split `document` the same way as Dataset 4.1.
2. `summary` is already a single sentence – no newline-splitting is needed for the gold reference here, unlike Dataset 4.1.

   Task 1 use: If assigned this dataset: build the TF-IDF sentence graph from `train`; tune k on `validation`; evaluate on `test`.

   Task 2 use: If assigned this dataset: train the pointer-generator on `train`; evaluate on `test` with Task 1's exact metrics.

   Task 3 use: If assigned this dataset: tune m on `validation`; run Task 1's extractor into Task 2's trained model on `test`; evaluate ROUGE and the factual-consistency proxy against Task 1 and Task 2, and discuss whether the pipeline helps less here than on a high-overlap dataset, given XSum's low source overlap.

Dataset 4.3 — SAMSum

Source

`https://huggingface.co/datasets/knkarthick/samsum` (a parquet-based mirror – **use this one, not the original** `Samsung/samsum` **repository**, whose legacy loading script breaks under recent versions of the `datasets` library). Load with `load_dataset("knkarthick/samsum")`. **License: CC-BY-NC-ND-4.0** – non-commercial use only, and no distribution of modified versions of the dataset itself; ordinary coursework use (training/evaluating a model, reporting results) is fine, but do not redistribute a modified copy of the dataset itself.

Format

Each example is one short, messenger-style dialogue between (usually) two speakers, paired with a third-person natural-language summary (e.g. "Amanda baked cookies and will bring some to Jerry tomorrow."). Unlike Datasets 4.1–4.2, the source is conversational and multi-speaker rather than a single-author article.

Field	Type	Meaning
`dialogue`	string	The full conversation: one turn per line, each line prefixed with a speaker name (e.g. `Amanda: I baked cookies.`), separated by newlines.
`summary`	string	A short, third-person summary of the conversation.
`id`	string	Dialogue identifier. Not a modeling feature.

Splits

Split	Size	Notes
`train`	14,732 dialogues	Fit TF-IDF/IDF statistics here.
`validation`	818 dialogues	Tune k, hyperparameters, and m here.
`test`	819 dialogues	Final reported metrics only.

Required Preprocessing

1. As Project 4's Task 1 states directly: treat each *utterance* (one speaker turn) as the selection unit here, not a sentence – split `dialogue` on its internal newlines to recover the list of individual turns.
2. Keep each turn's speaker-name prefix attached to that turn as part of the unit (do not strip it) – a selected turn like `Amanda: I baked cookies.` needs to keep "Amanda" attached for the extractive summary to remain interpretable as who said what; stripping speaker names would silently discard information the gold summaries themselves rely on (they are written in third person, explicitly naming speakers).
3. `summary` is already plain prose (not turn-separated) – use it as-is for both training (Task 2) and ROUGE scoring

(all three tasks).

Task 1 use: If assigned this dataset: build the TF-IDF turn-level graph from `train`; tune k on `validation`; evaluate on `test`.

Task 2 use: If assigned this dataset: train the pointer-generator on `train` (treating the newline-joined dialogue as the source sequence); evaluate on `test` with Task 1's exact metrics.

Task 3 use: If assigned this dataset: tune m on `validation`; run Task 1's extractor into Task 2's trained model on `test`; evaluate ROUGE and the factual-consistency proxy against Task 1 and Task 2, and discuss whether extract-then-abstract behaves differently on conversational, multi-speaker input than on single-author articles.

5 Project 5 Datasets: From Static to Contextual – Distributional Semantics and What's Inside BERT

Datasets 5.1–5.3 below are the three assignable options (each a UD treebank, supplying both raw text and gold dependency trees). Dataset 5.B (WordSim-353 + Google analogy set) is a separate, shared bonus check every group runs identically.

Dataset 5.1 — UD English EWT

Source, Format, Splits, Preprocessing

Identical to Dataset 2.1 (Section 2) – not repeated here. This project uses both the raw `tokens` (for building word vectors, Tasks 1 and 3) and the `head` field (for Task 2's structural-probe gold distances/depths) – the full schema already documented in Section 2.

Task 1 use: If assigned this dataset: build the word-word co-occurrence matrix and PPMI+SVD vectors from this dataset's train-split sentences (raw words only).

Task 2 use: If assigned this dataset: train/evaluate the structural probe on `train`/`test`, using gold tree-distance and tree-depth computed from `head`.

Task 3 use: If assigned this dataset: extract BERT-derived static vectors from this dataset's train-split sentences, using the layer identified in Task 2.

Dataset 5.2 — UD English GUM

Source, Format, Splits, Preprocessing

Identical to Dataset 2.2 (Section 2) – not repeated here. As Dataset 5.1, but for GUM – both raw text and gold dependency annotation from the schema in Section 2.

Task 1 use: If assigned this dataset: build the word-word co-occurrence matrix and PPMI+SVD vectors from this dataset's train-split sentences.

Task 2 use: If assigned this dataset: train/evaluate the structural probe on `train`/`test`.

Task 3 use: If assigned this dataset: extract BERT-derived static vectors from this dataset's train-split sentences.

Dataset 5.3 — UD English LinES

Source

`https://huggingface.co/datasets/universal_dependencies`, configuration `en_lines`. Load with `load_dataset("universal_dependencies", "en_lines")`.

Format

Identical CoNLL-U schema to Dataset 2.1 (same `tokens`/`head`/`deprel` fields) – a third, distinct English treebank originally built from translated/parallel text, giving a different style and vocabulary profile than EWT or GUM.

Splits

Split	Size	Notes
`train`	≈3,000 sentences	Use for building vectors (Tasks 1, 3) and training the probe (Task 2).
`test`	≈900 sentences	Final evaluation for Task 2's UUAS; also the source of your 20-sentence bonus-check sample.

Required Preprocessing

1. Use the `tokens`/`head`/`deprel` fields exactly as in Dataset 2.1.
2. This treebank is smaller than EWT – expect your PPMI+SVD and BERT-derived vectors (Tasks 1, 3) to be built from noticeably less text than if trained on EWT or GUM; this is an accepted trade-off of the

one-dataset-per-group design (see the project's Problem Statement).

Task 1 use: If assigned this dataset: build the word-word co-occurrence matrix and PPMI+SVD vectors from this dataset's train-split sentences.

Task 2 use: If assigned this dataset: train/evaluate the structural probe on `train`/`test`.

Task 3 use: If assigned this dataset: extract BERT-derived static vectors from this dataset's train-split sentences.

Dataset 5.B (Bonus, shared across every group) — WordSim-353 and the Google Analogy Set

Source

WordSim-353: `https://huggingface.co/datasets/wordsim353`. Google analogy set: distributed alongside the original word2vec release, `https://github.com/tmikolov/word2vec` (file `questions-words.txt`).

Format

Two files bundled as one evaluation suite: WordSim-353 is word pairs with a human similarity judgment; the analogy set is 4-word analogy questions.

Field	Type	Meaning
`word1`, `word2`	string	The two words being judged (WordSim-353).
`similarity`	float	Human-judged similarity, 0–10 scale (WordSim-353).
line format `a b c d`	4 strings	One analogy question: "a is to b as c is to ?", gold answer d.

Splits

No split – both are pure evaluation sets (353 pairs, ≈19,544 questions), used in full by every group regardless of which of Datasets 5.1–5.3 they were assigned.

Required Preprocessing

1. Lowercase every word to match the vocabulary casing from your assigned dataset.
2. Drop any analogy question with an out-of-vocabulary word; report how many were dropped, since your vocabulary size depends on which treebank you were assigned.
3. Compute Spearman (not Pearson) correlation for WordSim-353 – the task only needs relative ordering, not the human 0–10 scale matched numerically.

Task 1 use: Compute WordSim-353 Spearman correlation and Google-analogy accuracy for the PPMI+SVD vectors.

Task 2 use: Not used (Task 2 is evaluated with UUAS/depth-correlation instead, a structural rather than similarity/analogy metric).

Task 3 use: Compute the identical two metrics for the BERT-derived vectors, same protocol as Task 1.

6 Project 6 Datasets: A Text Processing Pipeline for Code-Mixed, Real-World Text

Datasets 6.1–6.3 below are three disjoint partitions of the same underlying SemEval SentiMix corpus, so every group gets a genuinely self-contained, correctly-labeled Hindi-English code-mixed sentiment dataset. Dataset 6.B (Dakshina) is a separate, shared resource every group uses identically for normalization.

Dataset 6.1 — SemEval SentiMix, Partition A

Source

`https://ritual-uh.github.io/sentimix2020/` (original shared-task release) or `https://huggingface.co/datasets/festvox/sentimix_hi_en` (mirror – **verify current field names against whichever source you use**). Combine the shared task's original train+validation+test into one pool of ≈20,000 tweets before partitioning (next column).

Format

Each example is one code-mixed tweet, tokenized, with a word-level language tag on every token and a single sentiment label for the whole tweet.

Field	Type	Meaning
`tokens`	list[string]	The tweet's words, already tokenized (mixed Hindi/English, Roman script throughout).
`lang_tag`	list[string]	Per-token language tag: `EN`, `HI`, or `UNIV`.
`sentiment`	string	Tweet-level label: `positive`, `negative`, or `neutral`.

Splits

Split	Size	Notes
Partition A / `train`	≈5,330 tweets	Roughly 80% of this partition's third of the combined pool.
Partition A / `validation`	≈670 tweets	Roughly 10%.
Partition A / `test`	≈670 tweets	Roughly 10%.

Required Preprocessing

1. **Construct all three partitions with one shared procedure** (do this once; use the identical seed across the whole class): combine the original train+validation+test into one pool, shuffle once with a fixed seed, cut into three equal thirds – Partition A, B, C – then split each third 80/10/10 into its own train/validation/test.
2. Join `tokens` with single spaces for the TF-IDF/MuRIL pipelines, but keep the original token list (and aligned `lang_tag` list) available separately – Task 1's LID classifier is trained per-token.
3. Verify exact label spelling/casing against whichever copy you load, and normalize it once, consistently.

Task 1 use: If assigned this partition: train the LID classifier and Random Forest sentiment classifier on Partition A's train split; evaluate macro-F1 on its test split.

Task 2 use: If assigned this partition: fine-tune MuRIL on Partition A's train split; evaluate macro-F1 on its test split.

Task 3 use: If assigned this partition: fine-tune the LID-augmented MuRIL on Partition A's train split; evaluate macro-F1 on its test split.

Dataset 6.2 — SemEval SentiMix, Partition B

Source, Format, Splits, Preprocessing

Identical to Dataset 6.1's construction procedure (Section 6) – not repeated here. The second third of the same shuffled pool, further divided 80/10/10 into its own train/validation/test.

Task 1 use: If assigned this partition: train the LID classifier and Random Forest on Partition B's train split; evaluate macro-F1 on its test split.

Task 2 use: If assigned this partition: fine-tune MuRIL on Partition B's train split; evaluate macro-F1 on its test split.

Task 3 use: If assigned this partition: fine-tune the LID-augmented MuRIL on Partition B's train split; evaluate macro-F1 on its test split.

Dataset 6.3 — SemEval SentiMix, Partition C

Source, Format, Splits, Preprocessing

Identical to Dataset 6.1's construction procedure (Section 6) – not repeated here. The remaining third of the same shuffled pool, further divided 80/10/10 into its own train/validation/test.

Task 1 use: If assigned this partition: train the LID classifier and Random Forest on Partition C's train split; evaluate macro-F1 on its test split.

Task 2 use: If assigned this partition: fine-tune MuRIL on Partition C's train split; evaluate macro-F1 on its test split.

Task 3 use: If assigned this partition: fine-tune the LID-augmented MuRIL on Partition C's train split; evaluate macro-F1 on its test split.

Dataset 6.B (Shared resource, used identically by every group) — Dakshina (Hindi Transliteration Pairs)

Source

`https://github.com/google-research-datasets/dakshina`. Use only the `hi` (Hindi) portion's word-pair romanization lexicon (not its separate full-sentence transliteration data, which is not used here).

Format

Each entry pairs a native-script (Devanagari) Hindi word with one or more attested Romanized spellings, each with a frequency count.

Field	Type	Meaning
native word	string	The word in Devanagari script.
romanization(s)	string(s)	One or more attested Romanized spellings of that word.
count	int	How many times each romanization was observed.

Splits

Use the full word-pair lexicon (all of Dakshina's own splits combined) – this is a fixed, static reference dictionary, identical for every group regardless of which SemEval partition (6.1, 6.2, or 6.3) they were assigned.

Required Preprocessing

1. Build a Romanized-spelling → canonical-form lookup dictionary: for each native word, treat its most frequent romanization as canonical, and map every other attested romanization of that word to it.
2. This dictionary is a static rule-based lookup, never trained or fit to any group's tweet partition, so there is no risk of leaking test labels into it.

Task 1 use: Build the normalization dictionary from this lexicon; apply it to your assigned partition's text before TF-IDF feature extraction.

Task 2 use: Apply the same dictionary to your assigned partition's test text at evaluation time only (the additional check described in Project 6's Task 2), never during MuRIL's raw-text training pass.

Task 3 use: Apply the same dictionary as part of the full preprocessing chain, alongside the LID-augmentation step.

7 Project 7 Datasets: Do LLMs Know When They're Making Things Up?

Dataset 7.1 — WikiBio GPT-3 Hallucination Dataset

Source

`https://huggingface.co/datasets/potsawee/wiki_bio_gpt3_hallucination` (from the SelfCheckGPT paper, Manakul et al., 2023).

Format

Each example is one GPT-3-generated, Wikipedia-style biography passage, split into sentences, with a human accuracy annotation per sentence, plus 20 independently resampled passages from the same prompt.

Field	Type	Meaning
`gpt3_text`	string	The full generated biography passage.
`gpt3_sentences`	list[string]	The passage split into individual sentences.
`annotation`	list[string]	Per-sentence label: `accurate`, `minor_inaccurate`, or `major_inaccurate`.
`gpt3_text_samples`	list[string]	20 independently resampled full passages (temperature 1.0), used by SelfCheckGPT.

Splits

Split	Size	Notes
full dataset	≈238 passages	No official split. If assigned this dataset, build your own fixed 80/20 train/test partition (stated seed) for Task 3's meta-classifier; Tasks 1–2 use the full dataset directly (their scoring methods are formulas, not trained models).

Required Preprocessing

1. Binarize the 3-way `annotation` label to 2-way for AUC-PR: `accurate` = negative, both `minor_inaccurate`/`major_inaccurate` = positive.
2. Keep `gpt3_sentences` and `annotation` index-aligned.

Task 1 use: If assigned this dataset: score every sentence with mean log-probability/entropy; report AUC-PR on the full dataset.

Task 2 use: If assigned this dataset: score every sentence with SelfCheck-BERTScore/NLI against the 20 resamples; report AUC-PR on the full dataset.

Task 3 use: If assigned this dataset: build your 80/20 split; train the meta-classifier on the 80%; evaluate AUC-PR on the held-out 20%.

Dataset 7.2 — TruthfulQA

Source

`https://huggingface.co/datasets/truthful_qa`, configuration `generation`. Load with `load_dataset("truthful_qa", "generation")`.

Format

Each example is one question designed to elicit a common misconception, with acceptable correct answers and known-incorrect answers.

Field	Type	Meaning
`question`	string	The question, often targeting a common misconception.
`correct_answers`	list[string]	Acceptable correct answers.
`incorrect_answers`	list[string]	Known-incorrect, plausible-sounding answers.
`category`	string	Topic category (38 categories total).

Splits

Split	Size	Notes
`validation`	817 questions	TruthfulQA's entire usable dataset. If assigned this dataset, build your own fixed 80/20 train/test partition for Task 3, same convention as Dataset 7.1.

Required Preprocessing

1. Generate one free-text answer per question with your chosen open LLM, fixed decoding configuration.
2. Score the generated answer accurate if it matches/closely paraphrases any `correct_answers` entry, inaccurate otherwise – state your matching method (similarity threshold, or hand-verified sample) explicitly.

Task 1 use: If assigned this dataset: generate an answer per question, score with log-probability/entropy; report AUC-PR on the full dataset.

Task 2 use: If assigned this dataset: generate $N = 20$ resamples per question; apply SelfCheck-BERTScore/NLI; report AUC-PR.

Task 3 use: If assigned this dataset: build your 80/20 split; train the meta-classifier on the 80%; evaluate AUC-PR on the held-out 20%.

Dataset 7.3 — HaluEval (QA Subset)

Source

`https://github.com/RUCAIBox/HaluEval` (Li et al., 2023); **confirm the exact split name for the QA subset against whichever mirror you use**.

Format

Each example pairs one question with a correct answer and an LLM-generated plausible-but-wrong answer.

Field	Type	Meaning
`question`	string	The question (originally drawn from an existing QA dataset).
`right_answer`	string	The correct answer.
`hallucinated_answer`	string	An LLM-generated plausible-sounding incorrect answer.

Splits

Split	Size	Notes
QA subset	≈10,000 pairs	No official split. If assigned this dataset, draw a fixed random sample for tractability if needed, and build your own 80/20 train/test partition for Task 3, same convention as Datasets 7.1–7.2.

Required Preprocessing

1. Score `right_answer` and `hallucinated_answer` directly with forced decoding (both are already given, unlike Datasets 7.1–7.2 where you generate an answer yourself).
2. Treat each pair as two separate scoring instances for AUC-PR purposes.

Task 1 use: If assigned this dataset: score both candidate answers per pair with log-probability/entropy; report AUC-PR.

Task 2 use: If assigned this dataset: generate resamples and apply SelfCheck-BERTScore/NLI (state your exact resampling protocol for this dataset); report AUC-PR.

Task 3 use: If assigned this dataset: build your 80/20 split; train the meta-classifier on the 80%; evaluate AUC-PR on the held-out 20%.

8 Project 8 Datasets: Chain-of-Thought Faithfulness

Dataset 8.1 — GSM8K

Source

`https://huggingface.co/datasets/openai/gsm8k`, configuration `main`.

Format

Each example is one grade-school arithmetic word problem, with a worked solution ending in a final numeric answer.

Field	Type	Meaning
`question`	string	The word problem.
`answer`	string	A worked solution, final numeric answer after a `####` delimiter.

Splits

Split	Size	Notes
`train`	7,473 problems	If assigned this dataset: source pool for 4-shot exemplars only, never used for training a model.
`test`	1,319 problems	Sample $n = 200$ with a fixed seed for all evaluation in Tasks 1–3.

Required Preprocessing

1. Extract the gold final numeric answer by splitting `answer` on `####`.
2. If using 4-shot prompting, draw the 4 exemplars from `train` with a fixed seed and reuse the same 4 for every question.
3. Strip currency symbols/commas before numeric comparison.

Task 1 use: If assigned this dataset: sample n=200 from `test`; evaluate direct-answer accuracy (zero-shot and 4-shot).

Task 2 use: If assigned this dataset: same sample; evaluate standard-CoT accuracy and all four faithfulness interventions.

Task 3 use: If assigned this dataset: same sample; evaluate the faithfulness-weighted self-consistency pipeline.

Dataset 8.2 — BIG-Bench-Hard Subtask

Source

`https://huggingface.co/datasets/maveriq/bigbenchhard`, configuration equal to your chosen subtask (e.g. `logical_deduction_five_objects`). If assigned this dataset, pick one subtask and use it consistently across all three tasks.

Format

Each example is one multiple-choice reasoning question with answer options embedded in the question text.

Field	Type	Meaning
`input`	string	The full question text, including embedded `(A)/(B)/...` options.
`target`	string	The correct option's text.

Splits

Split	Size	Notes
full subtask	typically 150–250 examples	Sample $n = 150$ with a fixed seed (or use the full subtask if smaller).

Required Preprocessing

1. Parse the embedded options out of `input` with a consistent regular expression.
2. Grade by matching the model's selected option *letter* against the letter corresponding to `target`.

Task 1 use: If assigned this dataset: sample n=150; evaluate direct-answer accuracy (zero-shot and 4-shot).

Task 2 use: If assigned this dataset: same sample; evaluate standard-CoT accuracy and all four faithfulness interventions.

Task 3 use: If assigned this dataset: same sample; evaluate the faithfulness-weighted self-consistency pipeline.

Dataset 8.3 — StrategyQA

Source

`https://huggingface.co/datasets/wics/strategy-qa`.

Format

Each example is a yes/no question whose reasoning steps are never stated explicitly – answering requires implicitly chaining multiple facts.

Field	Type	Meaning
`question`	string	The yes/no question (implicit multi-hop reasoning required).
`answer`	bool	The gold yes/no answer.
`facts` / `decomposition`	list[string]	Supporting facts used to construct the question. **Do not show these to the model at inference time.**

Splits

Split	Size	Notes
`train`	≈2,290 questions	If assigned this dataset: only source of few-shot exemplars, never used for training a model.
no official test split	–	Sample $n = 150$ from `train` with a fixed seed, disjoint from any few-shot exemplars.

Required Preprocessing

1. Confirm the exact boolean encoding used by whichever mirror you load, map to a consistent `Yes`/`No` string.
2. Sample your evaluation set and any few-shot exemplars from disjoint index sets within `train`.

Task 1 use: If assigned this dataset: sample n=150 (disjoint from few-shot exemplars); evaluate direct-answer accuracy (zero-shot and 4-shot).

Task 2 use: If assigned this dataset: same sample; evaluate standard-CoT accuracy and all four faithfulness interventions.

Task 3 use: If assigned this dataset: same sample; evaluate the faithfulness-weighted self-consistency pipeline.

9 Project 9 Datasets: Parameter-Efficient Fine-Tuning – From Hand-Engineered Features to LoRA

Dataset 9.1 — CoNLL-2003 (NER)

Source, Format, Splits, Preprocessing

Identical to Dataset 2.3 (Section 2) – not repeated here. Used identically to Section 2's description – same splits, same fields, same entity-level F1 protocol; only the models being trained on it differ (classical hand-engineered features and LoRA here, vs. HMM/BiLSTM-CRF in Project 2).

Task 1 use: If assigned this dataset: train the CRF-with-hand-crafted-features model on `train`; evaluate entity-level F1 on `test`.

Task 2 use: If assigned this dataset: fine-tune LoRA-adapted BERT (every rank r) on `train`; evaluate entity-level F1 on `test`.

Task 3 use: If assigned this dataset: train/evaluate the combined meta-model identically on the same `train`/`test` split.

Dataset 9.2 — GLUE SST-2

Source

`https://huggingface.co/datasets/nyu-mll/glue`, configuration `sst2`.

Format

Each example is one short, single-sentence movie-review snippet with a binary sentiment label.

Field	Type	Meaning
`sentence`	string	A short review snippet.
`label`	int	0 = negative, 1 = positive.
`idx`	int	Original GLUE example index. Not a modeling feature.

Splits

Split	Size	Notes
`train`	67,349 examples	Train here.
`validation`	872 examples	**Use this as your evaluation split.** GLUE's official `test` split ships with `label = -1` (labels withheld for the public leaderboard) and is not usable locally.

Required Preprocessing

1. Do not attempt to evaluate on `test` – check for `label = -1` after loading, which signals the label-withheld split.
2. Sentences here are much shorter than Dataset 9.3's full reviews – reconsider your TF-IDF n-gram range/feature cap accordingly and state your choice.

Task 1 use: If assigned this dataset: train TF-IDF + Random Forest on `train`; evaluate accuracy on `validation`.

Task 2 use: If assigned this dataset: fine-tune LoRA-adapted BERT (every rank r) and the full-fine-tuning reference on `train`; evaluate accuracy on `validation`.

Task 3 use: If assigned this dataset: train/evaluate the combined meta-model identically on `train`/`validation`.

Dataset 9.3 — IMDB

Source

`https://huggingface.co/datasets/stanfordnlp/imdb`.

Format

Each example is one full-length movie review with a binary sentiment label – entire paragraphs, unlike Dataset 9.2's short snippets.

Field	Type	Meaning
`text`	string	The full review text (may include leftover ` ` tags).
`label`	int	0 = negative, 1 = positive.

Splits

Split	Size	Notes
`train`	25,000 reviews	Balanced 12,500/12,500. Train here.
`test`	25,000 reviews	Also balanced, with real labels (unlike Dataset 9.2's) – use directly for final evaluation.

Required Preprocessing

1. Strip leftover `<br />` tags before tokenizing.
2. For LoRA/full-fine-tuning (Task 2), truncate reviews to your transformer's max sequence length (typically 512 subwords) and report what fraction of reviews you had to truncate.

Task 1 use: If assigned this dataset: train TF-IDF + Random Forest on `train`; evaluate accuracy on `test`.

Task 2 use: If assigned this dataset: fine-tune LoRA-adapted BERT (every rank r) and the full-fine-tuning reference on `train`; evaluate accuracy on `test`.

Task 3 use: If assigned this dataset: train/evaluate the combined meta-model identically on `train`/`test`.

10 Project 10 Datasets: Open-Domain Question Answering – From Lexical Retrieval to RAG

Dataset 10.1 — rag-mini-wikipedia

Source

`https://huggingface.co/datasets/rag-datasets/rag-mini-wikipedia`. Has two configurations: `text-corpus` (the passage index) and `question-answer` (the QA pairs) – load both.

Format

A small, self-contained Wikipedia passage collection paired with question-answer pairs written against that same collection – every question is guaranteed answerable from a passage actually present in the index.

Field	Type	Meaning
`passage`	string	One Wikipedia passage (from the `text-corpus` config).
`id`	int/string	Passage identifier.
`question`	string	A question (from the `question-answer` config).
`answer`	string	The gold short answer.

Splits

Split	Size	Notes
passage corpus	a few thousand passages	Index this in full with both BM25 and the dense retriever – this is the retrieval target for all three tasks.
QA pairs	≈900–1,000 questions	Use for all evaluation in Tasks 1–3.

Required Preprocessing

1. Build a BM25 index and a dense index over the same passage corpus, without re-chunking differently between the two.
2. For extractive scoring (Task 1), match the gold `answer` via SQuAD-style normalization (lowercase, strip articles/punctuation).

Task 1 use: If assigned this dataset: retrieve top-k passages with BM25; extract an answer span heuristically; evaluate EM/F1 and recall@k.

Task 2 use: If assigned this dataset: retrieve top-k passages with the dense retriever; generate an answer; evaluate EM/F1 and recall@k.

Task 3 use: If assigned this dataset: retrieve with the RRF-fused ranking; apply the groundedness check; evaluate EM/F1, recall@k, and groundedness-check accuracy.

Dataset 10.2 — Natural Questions (Open)

Source

`https://huggingface.co/datasets/google-research-datasets/nq_open`.

Format

The open-domain variant of Natural Questions: question and short answer(s) only, no given passage/context – the system itself is responsible for finding evidence.

Field	Type	Meaning
`question`	string	A real, anonymized Google search query.
`answer`	list[string]	One or more acceptable short answer strings.

Splits

Split	Size	Notes
`train`	≈87,900 questions	If assigned this dataset: source pool for generator few-shot exemplars only, never used for training a retriever/model.
`validation`	≈3,600 questions	Sample a fixed subset (e.g. $n = 300$, stated seed) for all evaluation in this project.

Required Preprocessing

1. **Build your own Wikipedia-paragraph index** covering your sampled questions' topics – NQ does not ship its own small passage index the way Dataset 10.1 does. If your group is assigned this dataset, this index is yours alone to build and use for all three tasks; document exactly how you built it (source dump, paragraph segmentation, size) in your report.
2. Score EM/F1 by comparing against *every* string in the `answer` list and taking the best match.

Task 1 use: If assigned this dataset: retrieve top-k passages from your constructed index with BM25; extract an answer heuristically; evaluate EM/F1.

Task 2 use: If assigned this dataset: retrieve with the dense retriever over the same index; generate an answer; evaluate EM/F1.

Task 3 use: If assigned this dataset: retrieve with the RRF-fused ranking; apply the groundedness check; evaluate EM/F1 and groundedness-check accuracy.

Dataset 10.3 — TriviaQA (`rc.nocontext`)

Source

`https://huggingface.co/datasets/mandarjoshi/trivia_qa`, configuration `rc.nocontext` (strips the bundled evidence documents TriviaQA's reading-comprehension setting normally provides, leaving bare question-answer pairs for the open-retrieval setting).

Format

Each example is one trivia-style question – often requiring more indirect or multi-hop reasoning than Natural Questions' search-query-style questions – with a gold answer and accepted alternate phrasings.

Field	Type	Meaning
`question`	string	A trivia-style question.
`answer`	dict	Contains `value` (canonical gold answer) and `aliases` (accepted alternate phrasings).

Splits

Split	Size	Notes
`train`	≈78,800 questions	If assigned this dataset: source pool for few-shot exemplars only, never used for training.
`validation`	≈8,800 questions	Sample a fixed subset (e.g. $n = 300$, stated seed).

Required Preprocessing

1. **Build your own Wikipedia-paragraph index**, the same way as Dataset 10.2 – if your group is assigned *this* dataset instead of Dataset 10.2, you will not have an NQ index to reuse, so build one from scratch covering your sampled TriviaQA questions' topics, and document how.
2. Combine `answer['value']` and every entry in `answer['aliases']` into one flat list of acceptable strings before scoring EM/F1.

Task 1 use: If assigned this dataset: retrieve top-k passages with BM25 from your constructed index; extract an answer heuristically; evaluate EM/F1.

Task 2 use: If assigned this dataset: retrieve with the dense retriever; generate an answer; evaluate EM/F1.

Task 3 use: If assigned this dataset: retrieve with the RRF-fused ranking; apply the groundedness check; evaluate EM/F1 and groundedness-check accuracy.

11 Project 11 Datasets: Metaphor Detection – Selectional-Preference Violation vs. Neural Token Classification

Dataset 11.1 — VU Amsterdam Metaphor Corpus (VUA), All-POS Track

Source

Official shared-task release: `https://github.com/EducationalTestingService/metaphor/tree/master/VUA-shared-task` (2018 VUA Metaphor Detection Shared Task, all-POS track). Distributed as per-token TSV/CSV files, organized by document and by the four source registers (news, academic, fiction, conversation) drawn from the BNC Baby corpus.

Format

Every content word in every sentence is labeled, under the MIPVU annotation protocol, as metaphorical or literal – this is a full *sequence labeling* task (one label per token), unlike Datasets 11.2–11.3 below, which label only a single target verb per sentence. The shared-task release already includes gold POS tags; it does **not** include dependency parses – you will need to parse the sentences yourself for Task 1's predicate-argument extraction.

Field	Type	Meaning
`txt_id`	string	Source document identifier (117 documents total).
`sentence_id`	int	Sentence index within the document.
`token`	string	The word form.
`pos`	string	Gold part-of-speech tag (provided by the release).
`label`	int	1 = metaphorical, 0 = literal, under the MIPVU protocol.
`genre`	string	One of `news`, `academic`, `fiction`, `conversation`.

Splits

Split	Size	Notes
`train`	≈96 documents	The shared task's official train documents. Hold out ≈15% of these documents (not individual tokens – whole documents, to avoid leaking a document's vocabulary/style across the split) as your validation set, with a fixed seed.
`test`	≈21 documents	The shared task's official held-out test documents – use directly, do not re-split. *(Exact per-document and per-token counts vary slightly by release version; confirm against the copy you download.)*

Required Preprocessing

1. Group tokens back into sentences using `txt_id` + `sentence_id` before running a dependency parser (e.g. spaCy) over each sentence – Task 1's selectional-preference computation needs subject/object relations, which this release does not provide directly.
2. Split validation from train *by whole document*, not by shuffling individual tokens or sentences – documents in this corpus are long and stylistically consistent, so a token-level random split would leak a document's specific vocabulary between train and validation.
3. Keep the `genre` field even though it is not required by any task step here – it is useful if you want to sanity-check whether your model's errors cluster in a particular register in your write-up.

Task 1 use: If assigned this dataset: build selectional-preference profiles and tune the association-score threshold on train/validation; evaluate precision/recall/F1 (overall and by POS) on test.

Task 2 use: If assigned this dataset: train the BiLSTM sequence labeler (one label per token) on train, early-stopping on validation F1; evaluate on test with Task 1's exact metrics.

Task 3 use: If assigned this dataset: compute the Task 1/Task 2 correlation and retrain the feature-augmented model on train/validation; evaluate on test, including the disagreement-subset breakdown.

Dataset 11.2 — TroFi (Trope Finder)

Source

Direct file: `http://www.cs.sfu.ca/~anoop/students/jbirke/TroFiExampleBase.txt`, or the identical mirror at `https://github.com/sfu-natlang/trofi-metaphor-data/blob/master/TroFiExampleBase.txt`. A single plain-text file – no account, form, or installer needed. **License: GPL**; if you use this data, cite Birke & Sarkar, *Clustering Approach for the Nearly Unsupervised Recognition of Nonliteral Language*, EACL 2006, and Birke & Sarkar, *Active Learning for the Identification of Nonliteral Language*, NAACL-HLT 2007 workshop, as required by the release.

Format

Built around 50 fixed, ambiguous English verbs (e.g. *absorb*, *grasp*, *strike* – the full list of 50 is on the source page), organized as text blocks per verb, each split into a *nonliteral cluster* and a *literal cluster* section. Each line inside a cluster has three tab-separated columns: a WSJ source identifier, an annotation-status label, and the tokenized sentence (ending in `./.`). **This is not the clean `sentence/verb_idx/label` table a typical dataset release would give you** – it needs to be parsed out of this block structure yourself.

Field	Type	Meaning
WSJ id	string	Source sentence identifier, e.g. `wsj02:2251` (file number : line number, across 86 WSJ source files).
annotation label	string	One of `L` (Literal), `N` (Nonliteral), or `U` (Unannotated) – see Required Preprocessing.
tokenized sentence	string	The WSJ sentence, already tokenized, ending in `./.`

Splits

No official train/validation/test split; construct your own. **First**, keep only `L`- and `N`-labeled lines – discard every `U` (unannotated) line, since there is no ground truth for those. **Then**, pool the remaining labeled sentences across all 50 verbs, shuffle once with a fixed seed, and take 80%/10%/10% as train/validation/test by *sentence* (not by verb), so every split contains most/all 50 verbs rather than a split where some verbs never appear in training.

Required Preprocessing

1. Parse the raw file into individual records: for each verb block (delimited by header lines like `***absorb***`), read the nonliteral and literal cluster sub-sections separately, recording per line the verb (from the block header), the WSJ id, the L/N/U label, and the tokenized sentence.
2. **Discard every `U`-labeled line before splitting** – only `L` and `N` lines have a real gold label (`N` → label 1/metaphorical, `L` → label 0/literal).
3. Locate the target verb's token index within each sentence by matching the verb string, allowing for simple inflected

forms (e.g. *absorbs*, *absorbed*) against the block header's base form.

4. POS-tag and dependency-parse each sentence yourself – this release provides neither.
5. **Deviation from standard practice – state this explicitly in your report:** most published work on TroFi (e.g. Gao et al., MelBERT) evaluates it with k-fold cross-validation rather than a single fixed split, precisely because the dataset is small and organized per-verb. This project instead uses a single fixed 80/10/10 split, for consistency with how every other dataset in this document is handled. Results here are therefore **not** directly comparable to published TroFi numbers, and this should be stated plainly in your report rather than presented as a like-for-like comparison.

Task 1 use: If assigned this dataset: build selectional-preference profiles and tune the threshold on train/validation (scoring only each sentence's target verb); evaluate precision/recall/F1 on test.

Task 2 use: If assigned this dataset: train the target-word-only classifier (not full sequence labeling) on train, early-stopping on validation F1; evaluate on test with Task 1's exact metrics.

Task 3 use: If assigned this dataset: compute the Task 1/Task 2 correlation and retrain the feature-augmented model; evaluate on test, including the disagreement-subset breakdown.

Dataset 11.3 — MOH-X

Source

No official standalone release page. The cleanest source is the pre-formatted bundle distributed by Gao et al. alongside their paper's code: Google Drive link `https://drive.google.com/file/d/1-v_sUlupDrKq8ERlnh5RwdI7Pyk-6cxn/view?usp=sharing` (linked from `https://github.com/gao-g/metaphor-in-context`), which bundles pre-formatted versions of MOH-X, TroFi, and VUA together – download and unzip this once as a group; there is no pip/CLI installer for it. If that link ever goes stale, the MelBERT repository links to an equivalent bundle.

Format

A small, curated subset of the original Mohammad et al. MOH dataset, restricted to sentences with a single, unambiguous target verb. The bundle above has already been cleaned into a simple per-example format (unlike Dataset 11.2's raw block structure) – **exact column names may vary slightly by release version, since this is an unofficial convenience bundle rather than a versioned dataset release; inspect the first few rows after unzipping and adjust your loader to match.** The description below gives the logical fields you should end up with regardless of the exact on-disk column names.

Field	Type	Meaning
sentence	string	The full sentence containing the target verb.
verb	string	The target verb.
verb_idx	int	Token index of the target verb within the sentence.
label	int	1 = metaphorical, 0 = literal, for this one verb in this one sentence.

Splits

No official split (this dataset is conventionally used with k-fold cross-validation in the papers that introduced and later used it – see Dataset 11.2's deviation note, which applies identically here). If assigned this dataset, construct your own fixed-seed 80/10/10 train/validation/test split by sentence, the same procedure as Dataset 11.2, and state the same deviation from standard practice in your report. **Because the full dataset is only a few hundred sentences**, your resulting test split will be very small (on the order of a few dozen sentences) – report F1 with this sample size explicitly in mind.

Required Preprocessing

1. Tokenize, POS-tag, and dependency-parse every sentence yourself, as with Dataset 11.2.
2. Given the small size, consider reporting results averaged over a few different random splits (with different seeds) rather than a single fixed split, and note the spread across runs – a single split's F1 here can be noisy purely from sample size, independent of model quality.

Task 1 use: If assigned this dataset: build selectional-preference profiles and tune the threshold on train/validation; evaluate precision/recall/F1 on test, reporting variance across at least 3 random splits given the small sample size.

Task 2 use: If assigned this dataset: train the target-word-only classifier on train, early-stopping on validation F1; evaluate on test with Task 1's exact metrics and the same multi-split variance reporting.

Task 3 use: If assigned this dataset: compute the Task 1/Task 2 correlation and retrain the feature-augmented model; evaluate on test including the disagreement-subset breakdown, and discuss whether the small sample size makes the disagreement-subset analysis reliable at all.